

סימולטור Production Trap

גשר בין האקדמיה למציאות התעשייתית

מגישות: יעל בוכמן 038151114 וליאור קץ 318909496

1. מבוא ותקציר מנהלים

בעידן שבו כלי בינה מלאכותית (LLMs) הופכים לחלק בלתי נפרד מסביבת העבודה של מפתחי תוכנה, עולה הצורך בשינוי פרדיגמה בחינוך הטכנולוגי. פרויקט "Production Trap" נועד להתמודד עם אתגר זה על ידי יצירת סימולציית "מחולל בעיות" – סביבה דינמית המדמה את העמימות, הלחצים והמורכבות של סביבת ייצור (Production) אמיתית.

הסימולטור אינו בוחן רק כתיבת קוד תקין, אלא את היכולת של הסטודנט להבין קונטקסט מערכתי, לנהל משברים ולפתור תקלות שאינן מופיעות בספרי הלימוד. באמצעות שימוש בארכיטקטורת Multi-Agent מבוססת LangGraph, המערכת מייצרת חוויה אישית, חסינה להעתקות (Anti-Cheat) וברת-הרחבה (Scale).

כל המימוש נמצא ב- <https://github.com/Yael-Bu/SeminarFinalAgent>

סרטון הסבר והדגמה נמצא ב- <https://github.com/Yael-Bu/SeminarFinalAgent/blob/main/Seminar.mov>

2. חזון פדגוגי: הלמידה דרך ה"מלכודת"

לב הפרויקט טמון ביצירת פער מכון בין שלב הפיתוח (Development) לשלב הייצור (Production).

- **הסחת דעת פונקציונלית:** הסטודנט מקבל משימה שנראית פשוטה לביצוע. המטרה היא לגרום לו להתמקד ב"מה" (הפונקציונליות) ולזנוח את ה"איך" (יעילות, חסינות, עומסים).
- **כישלון ככלי לימודי:** בניגוד לתרגילים קלאסיים שבהם הקוד פשוט "לא עובד בדיקות", כאן הקוד מאושר על ידי ה-Team Lead וקורס רק ב"שטח". חוויה זו מדמה את רגעי הלחץ האמיתיים בתעשייה ומפתחת חוסן מנטלי ושיקול דעת מקצועי.

3. ארכיטקטורת המערכת וניהול הגרף (Multi-Agent System)

המערכת מבוססת על ספריית LangGraph, המאפשרת ניהול מצבים (State Management) מורכב וניתוב דינמי בין סוכני AI.

א. ניהול המצב (SimulationState)

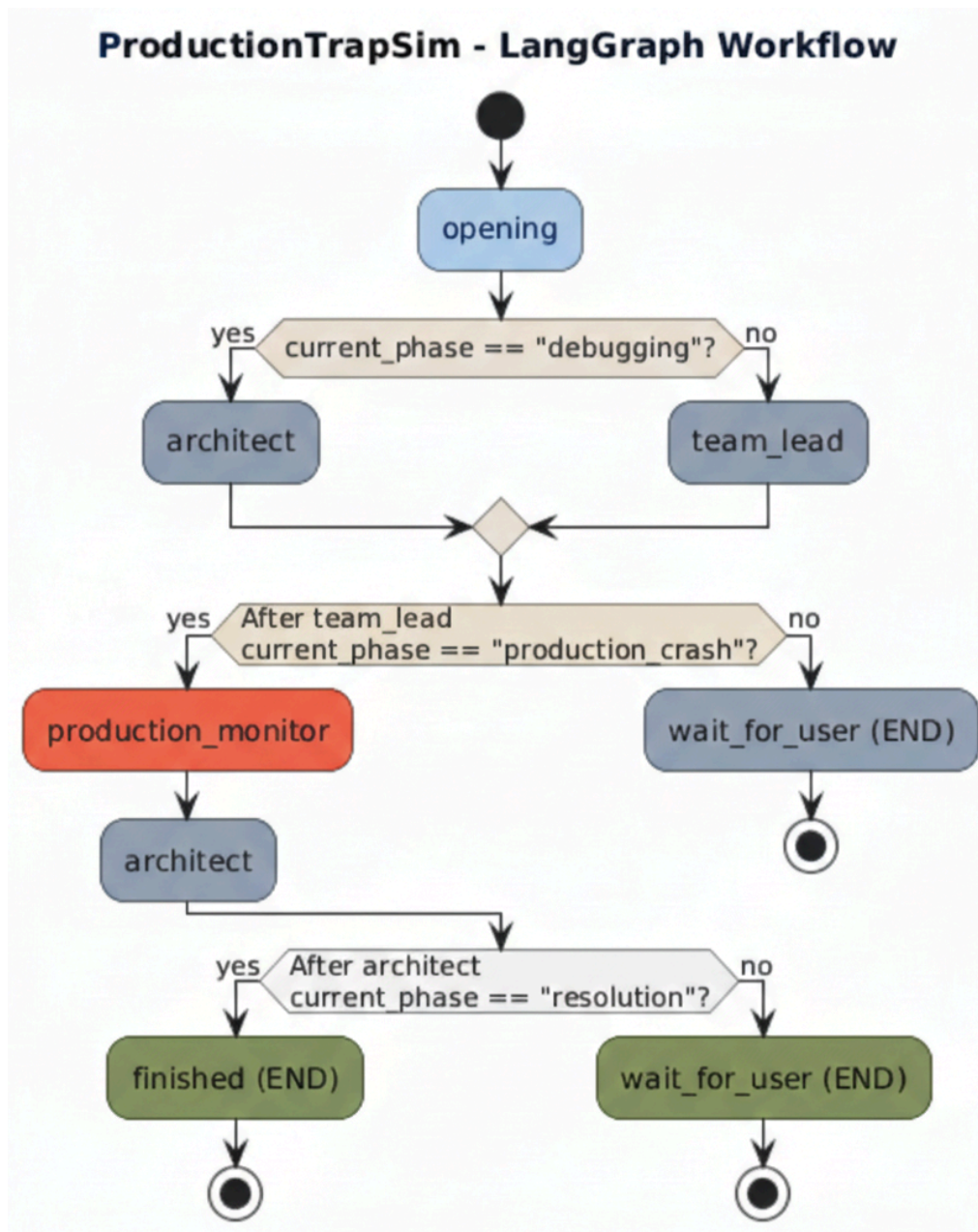
המערכת שומרת "זיכרון" של שלבי הסימולציה באובייקט `SimulationState`:

- **messages:** היסטוריית השיחה המלאה. המערכת משתמשת במנגנון `add_unique_messages` למניעת כפילויות בהדפסה ובעיבוד.

- **current_phase**: השלב הנוכחי (development, production_crash, debugging, resolution), המכתיב את הניתוב בגרף.
- **scenario_data**: הנתונים הדינמיים של התרחיש שנוצר עבור הסטודנט.

ב. זרימת הגרף (The Graph Workflow)

הלב הפועם של הסימולטור הוא גרף המצבים המאפשר ניתוב דינמי:



דיאגרמה 1: תרשים זרימת הגרף (Graph Workflow)

הגרף אוסף את התהליך הפדגוגי: הסטודנט מתחיל מול ה-Team Lead, עובר ל-Production Monitor המדמה קריסה, ומשם נדרש לעבוד מול ה-Architect לתיקון הבעיה. הסטודנט אינו יכול "לדלג" על שלב הניתוח הטכני, שכן הגרף מחייב מעבר דרך צומת הארכיטקט והצגת פתרון העומד בקריטריונים מחמירים.

4. ניתוח פרסונות הסוכנים והנדסת הפרומפטים

א. פרסונה - קווים לדמותה

הסימולציה נשענת על יכולת הסוכנים לאמץ פרסונה מקצועית אמינה:

- **ה-Senior Team Lead (טמפרטורה: 0.3):** מעוצב כמנהל משימתי הלחץ לתוצאות. הוא בודק רק פונקציונליות בסיסית ("האם זה עובד?") ומגיב בביטוי "DEPLOYING TO PRODUCTION" בלבד במקרה של הצלחה, מה שמייצר אשליה של ביטחון אצל הסטודנט.
- **ה-Production Monitor (סוכן מערכת):** רכיב דטרמיניסטי המשמש כ"שופר" של מערכת הייצור. הוא מדווח על הסימפטום (למשל, IP Ban) אך לא על הסיבה (למשל, חוסר ב-Caching).
- **ה-Senior Architect (טמפרטורה: 0.3):** הסוכן המקצועי ביותר. הוא מונחה ל-**Strict Evaluation Scope** ובודק את הקוד מול רשימת דרישות (Checklist) וקריטריוני הצלחה קשיחים. הוא מספק רמזים טכניים בלבד מבלי לפתור את הבעיה עבור הסטודנט.

ב. אופטימיזציה טכנית של המודלים

1. בחירת מודלי השפה (LLM Selection)

בפרויקט נעשה שימוש מושכל בשני מודלים שונים ממשפחת OpenAI כדי לאזן בין ביצועים לעלות (Cost-Efficiency):

- **GPT-4o (הסוכנים):** נבחר עבור סוכני ה-Team Lead והארכיטקט. היכולת של מודל זה לנתח קוד (Reasoning) ולהבין לוגיקה עסקית מורכבת היא קריטית כדי שהארכיטקט יוכל לבצע וולידציה מדויקת מול דרישות ה-Checklist.
- **GPT-4o-mini (מחולל התרחישים):** נבחר עבור ה-ScenarioManager. מכיוון שיצירת תרחיש (Dynamic Skinning) היא משימה של עיבוד שפה טבעית (NLP) ולא של ניתוח קוד עמוק, השימוש במודל הקטן והמהיר יותר מאפשר סקילביליות (Scalability) גבוהה וחיסכון במשאבים מבלי לפגוע באיכות התוצאה לסטודנט.

2. אסטרטגיית הטמפרטורה (Temperature Strategy)

לאחר בדיקת מספר אופציות נקבעו הגדרות הטמפרטורה בהתאם לתפקיד של הסוכן:

- **טמפרטורה 0.3** נבחרה לסוכנים מצוות הפיתוח כדי לאפשר "גיוון אנושי" קל בניסוח המשימות והמשוב, אך מבלי לאבד את המיקוד המקצועי. זה מבטיח שהסוכן יישמע טבעי אך לא יחרוג מהדרישות המקוריות.
- **טמפרטורה 0.8** נבחרה ליצירת תרחישים מגוונים וייחודיים שאינם חוזרים על עצמם תחת האילוצים ההכרחיים.

5. מנגנון ה-Anti-Cheat וה-Scale: ה-Student Signature

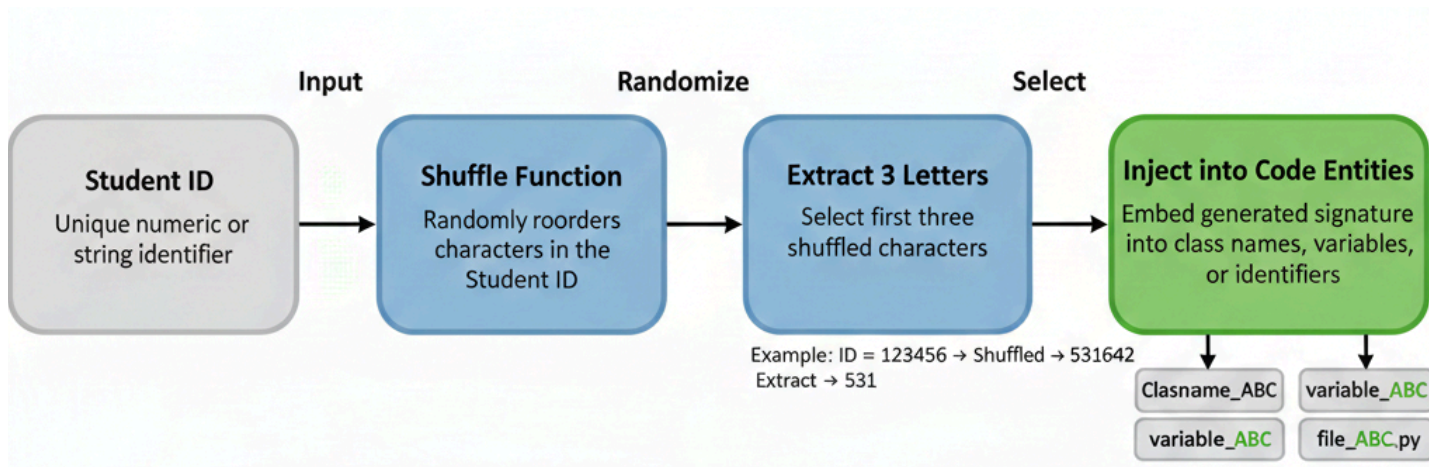
אחת הדרישות המרכזיות היא יצירת וריאציות ייחודיות לכל סטודנט.

א. תהליך יצירת החתימה (helper.py)

המערכת משתמשת בתעודת הזהות של הסטודנט כעוגן:

1. חילוף ספרות תעודת הזהות.

- ביצוע Shuffle (ערבול) לספרות באמצעות זרע (Seed) מבוסס ID להבטחת תוצאה עקבית אך לא צפויה.
- מיפוי 3 הספרות הראשונות לאותיות (A-J) ליצירת "חתימה" אלפביתית (למשל, HGA_).



דיאגרמה 2: תהליך יצירת ה-Student Signature

ב. הזרקה דינמית (ScenarioManager)

ה-ScenarioManager מזריק את החתימה לתוך התרחיש: כל שמות הטבלאות, הפונקציות וה-APIs מקבלים את הסיומת הייחודית. ה-Architect בודק שהסטודנט השתמש בדיוק בשמות הללו, מה שמונע העתקות מ-ChatGPT או מחברים.

6. בניית תרחיש: המתודולוגיה והשימוש בשדות

א. פירוט תרחישים קיימים (Test Cases)

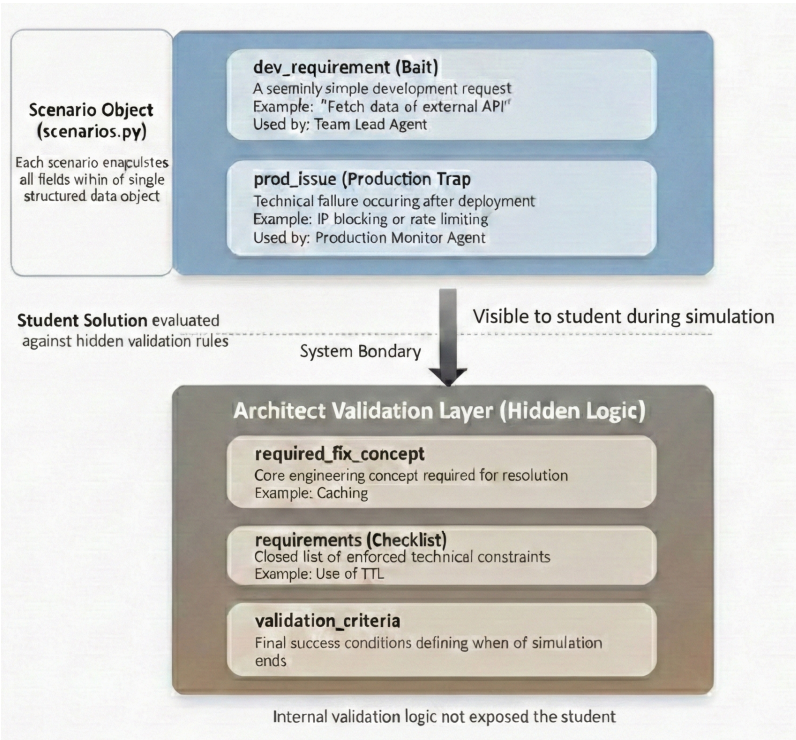
המערכת כוללת כיום ספריית תרחישים המכסה אתגרי פיתוח שונים. להלן ניתוח של שלושה תרחישי ליבה המופיעים בלוגים ובקוד:

שם התרחיש	הקונספט ההנדסי (Concept)	ה"מלכודת" (The Trap)	קריטריון הצלחה (Validation)
Legacy Token	תאימות לאחור (Backward Compatibility)	מערכות ישנות נחסמות בשל דרישת Token קשיחה.	החרגת לקוחות Legacy מה-Middleware מבלי לפגוע באבטחה.
Rate Limit	ניהול משאבים (Caching/TTL)	הצפה של API חיצוני מובילה לחסימת IP גלובלית.	הטמעת שכבת זיכרון (Cache) המונעת קריאות כפולות לאותו מיקום.

ביצוע עדכונים בקבוצות (Batching) או שימוש בתחביר שאינו חוסם.	הוספת עמודה עם ערך ברירת מחדל נועת את הטבלה לשימוש.	הגירה מקוונת (Online Migration)	DB Lock
--	--	---------------------------------	---------

ב. מבנה שדות התרחיש

בניית תרחיש מורכבת משני רבדים הנדסיים:



דיאגרמה 3: מבנה התרחיש (Scenario Data Structure)

כל תרחיש ב-scenarios.py מוגדר באמצעות שדות המזינים את הסוכנים בשלבים שונים:

- dev_requirement ("פיתיון"):** דרישה שנראית פשוטה (למשל: "משוך נתונים מ-API חיצוני"). משמשת את ה-Team Lead.
- prod_issue ("מלכודת"):** תיאור קטסטרופה טכנית המתרחשת לאחר ה-Deployment (למשל: חסימת IP). משמשת את ה-Production Monitor.
- required_fix_concept:** הקונספט ההנדסי הנדרש לתיקון (למשל: Caching). שדה זה מנחה את ה-Architect.
- requirements ("Checklist"):** רשימת דרישות טכניות סגורה שהארכיטקט אוכף (למשל: שימוש ב-TTL).
- validation_criteria:** קריטריוני ההצלחה הסופיים המגדירים מתי הסימולציה מסתיימת.

ב. דינמיות בזמן ריצה

המערכת משתמשת ב-LLM gpt-4o-mini כדי לבצע "הלבשה" (Skinning) לתרחיש. הבעיה הטכנית נשארת זהה, אך התפאורה והשמות משתנים בהתאם לחתימת הסטודנט, מה שהופך את האתגר למקורי בכל הרצה.

ג. קלות ההרחבה: הוספת תרחיש חדש (Plug-and-Play)

אחד מיתרונות המערכת הוא היכולת להוסיף תרחישים חדשים ללא שינוי בקוד הליבה של הסוכנים או הגרף. התהליך מתבצע בשלושה שלבים פשוטים בתוך קובץ ה-`scenarios.py`:

1. הגדרת האובייקט: יצירת מילון (Dictionary) חדש המכיל את שדות הליבה: `dev_requirement`, `prod_issue` ו-`requirements`.

2. קביעת ה-Signature: הגדרת הישויות (Entities) שיקבלו את חתימת הסטודנט הייחודית (למשל שמות טבלאות או פונקציות), מה שמבטיח הגנה מהעתקה גם בתרחיש החדש.

3. הזרקה ל-ScenarioManager: הוספת המילון לרשימת התרחישים הזמינים. המערכת תדע לבחור אותו באופן אקראי ולבצע לו "הלבשה" (Skinning) אוטומטית בזמן ריצה.

דוגמה לתרחיש עתידי: ניתן להוסיף בקלות תרחיש של "Memory Leak" שבו הסטודנט נדרש לממש עיבוד קבצים גדולים. המלכודת תופעל כאשר הקוד ינסה לטעון קובץ ענק לזיכרון במקום להשתמש ב-Streaming.

7. מימוש טכני וניהול חוסן (Logger & Robustness)

- מנגנון ה-Logger: הוטמע מנגנון תיעוד המבצע הדפסה לטרמינל וכתיבה לקובץ לוג ייחודי. שיטת ה-`log_only` מאפשרת תיעוד קוד המשתמש ללא "לכלוך" המסך.
- חוסן (Architect Graceful Failure): הארכיטקט כולל מנגנון `try-except`. במקרה של שגיאה טכנית במודל ה-AI, המערכת מוגדרת לעבור אוטומטית למצב `resolution` כדי להבטיח שהסטודנט יוכל לסיים את המשימה.
- מניעת כפילויות: שימוש ב-`already_printed_ids` מבטיח חוויית משתמש נקייה מול ה-`stream` של LangGraph.

8. מתודולוגיה ותהליך הפיתוח (Work Stages)

תהליך הפיתוח של "Production Trap" התבצע באופן איטרטיבי, תוך דגש על יצירת פער פדגוגי מכוון בין שלב הפיתוח לייצור. להלן השלבים המרכזיים שביצענו:

א. תכנון זרימת הקוד בגרף (Workflow Design):

כפי שנלמד בקורס, זיהינו כי לניהול מצבים מורכב ודינמי, הדרך הנכונה ביותר היא עבודה עם LangGraph. תכנונו את הגרף כך שידמה את ה"דרך שהקוד עובר" בתעשייה – מהפיתוח הראשוני, דרך אישור ה-Team Lead, ועד לקריסה בייצור והתערבות הארכיטקט.

ב. בניית שלד המערכת (MVP):

בתחילה בנינו גרף פשוט שבו לכל צומת הוגדר "פרומפט" בסיסי מאוד המגדיר את זהות הסוכן ותפקידו. בשלב זה פיתחנו שלושה תרחישים שונים (Test Cases) וביצענו הרצות ראשוניות כדי לוודא שהמערכת מצליחה לעבור בכל הקודקודים בגרף עבור כל אחת מהסימולציות.

ג. זיקוק סוכנים ופיגומים פדגוגיים (Scaffolding):

לאחר שהשלב עבר, עברנו לשיפור וחיזוק הפרומפטים. המיקוד העיקרי היה על סוכן ה-Architect. היינו צריכות למצוא את האיזון המדויק: מצד אחד, לשמור על קשיחות מקצועית שלא תאשר פתרונות שטחיים, ומצד שני, לנסח רמזים טכניים שיקדמו את הסטודנט מבלי לגלות לו את התשובה הסופית.

ד. פיתוח מנגנון הגנה מהעתקות (Anti-Cheat):

כדי להבטיח וריאציה ייחודית לכל משתמש, הוספנו רובד של הזרקה דינמית. השתמשנו ב-LLM (מודל gpt-4o-mini) כדי לבצע "הלבשה" (Skinning) לכל תרחיש בהתאם לחתימת הסטודנט. שלב זה כלל כתיבת לוגיקה שמזריקה שמות טבלאות ופונקציות ייחודיים לכל הרצה, מה שהופך כל אתגר למקורי ובלתי ניתן להעתקה גנרית.

9. בדיקות וקריטריונים להצלחה (Testing & Success Criteria)

כדי לוודא שהסימולטור עומד ביעדים הפדגוגיים והטכניים שהוגדרו, ביצענו סדרת בדיקות מקיפה. מטרת הבדיקות הייתה לוודא שהמערכת אינה רק "מריצה קוד", אלא מתפקדת ככלי לימודי ביקורתי וחסין להעתקות.

א. מה בדקנו? (Testing Scope) במסגרת תהליך הפיתוח, ביצענו בדיקות לכל אחד מרכיבי הגרף:

- בדיקת זרימה דטרמיניסטית (Graph Logic):** וידאנו שהמעבר בין הצמתים (Nodes) מתבצע רק לאחר עמידה בתנאי הסף (למשל: לא ניתן להגיע לארכיטקט ללא אישור מה-Team Lead וקריסה בייצור).
- דיוק הערכת הקוד (Agent Accuracy):** הזנו לארכיטקט פתרונות שונים – פתרונות נכונים, פתרונות חלקיים ופתרונות "גנריים" (ללא חתימת סטודנט). בדקנו האם הוא מצליח להבחין ביניהם ולתת משוב רלוונטי.
- חסינות מנגנון ה-Signature:** בדקנו האם המערכת פוסלת קוד שמשתמש בשמות טבלאות או פונקציות שאינם תואמים ל-ID של המשתמש הספציפי.
- בדיקת מקרי קצה (Edge Cases):** בדקנו כיצד המערכת מגיבה לניסיונות "פריצה" של הסטודנט, כגון הגשת קוד ריק או הודעות שאינן קשורות למשימה.

ב. קריטריונים להצלחה (Success Criteria) קבענו שלושה מדדים אובייקטיביים שרק עמידה בכלם נחשבת ל"סיום מוצלח" של הסימולציה:

- סטטוס SOLVED:** קבלת אישור סופי מהסוכן הארכיטקט על בסיס ה-Validation Criteria שהוגדרו מראש בתרחיש.
- אימות זהות הנדסי (Signature Match):** וידוא שכל הישויות הטכניות בקוד (Database Tables/Functions) כוללות את הסימט הייחודית של הסטודנט.
- תיקון הקונספט (Conceptual Fix):** הצלחה מוגדרת רק אם הסטודנט מימש את הקונספט הנדסי הנדרש (למשל: Batching במיגרציה) ולא רק פתרון שטחי ש"עוקף" את הבעיה.

ג. מתודולוגיית ניתוח התוצאות (Analysis Methodology) ניתוח התוצאות התבצע בשני מישורים:

1. **ניתוח כמותי:** מעקב אחר מספר האיטרציות (שיחות) שנדרשו לכל משתמש עד להגעה לפתרון, במטרה להעריך את אפקטיביות הרמזים של הארכיטקט.
2. **ניתוח איכותני (Log Analysis):** מעבר ידני על ה-Reasoning של הסוכנים כפי שתועד ב-Logger. בדקנו האם המשוב שניתן לסטודנט היה מקצועי, האם הוא קידם אותו לעבר הפתרון והאם הסוכן שמר על הפרסונה שלו לאורך כל התהליך.

10. ניתוח תוצרי ריצה והוכחת היתכנות (Log Analysis)

ניתוח של קובצי הלוג שנוצרו במהלך הרצות המערכת (הנמצאים בתיקיית Results במאגר: <https://github.com/Yael-Bu/SeminarFinalAgent/tree/main/Results>) מהווה הוכחה ליכולת המערכת לעמוד ביעדים הפדגוגיים והטכניים. להלן ניתוח של שלושה מקרי בוחן מייצגים:

א. חסינות ל-AI ופיגומים פדגוגיים (Scaffolding)

קובץ רלוונטי: simulation_log_20260215_134725.txt

בלוג זה ניתן לראות כיצד הסוכן מתפקד כבודק איכות (Auditor) קפדני. המערכת אינה מאשרת פתרון המבוסס על "קוד שעובד" בלבד, אלא דורשת חוסן (Resilience) הנדסי.

- **הוכחה מהלוג:** הסטודנט הגיש קוד לוגי תקין למיגרציה, אך הסוכן פסל אותו שלוש פעמים בגלל חוסר בטיפול בשגיאות (Error Handling) וב-Retries לחיבור.
- **ציטוט מייצג:** Agent: ❌ Add error handling for the backfill_in_batches function.
- **תובנה:** המערכת מאלצת את הסטודנט לחרוג מהפתרון הגנרי של ChatGPT ולהתמודד עם כשלים שקורים רק בעולם האמיתי.

ב. אתגר התאימות לאחור (Backward Compatibility)

קובץ רלוונטי: simulation_log_20260215_140531.txt

תרחיש זה מדגים את ה"מלכודת" בשיאה. הקוד שאושר בשלב הפיתוח קורס בייצור בשל קונטקסט עסקי שהסטודנט לא לקח בחשבון.

- **הוכחה מהלוג:** לאחר פריסת Middleware מאובטח, המערכת מדווחת על קריסה של מערכות הדיווח של הנהלת החברה.
- **ציטוט מייצג:** Agent: 🚨 PRODUCTION ALERT 🚨 ⚡ CRITICAL INCIDENT: Legacy Reporting !System_DCB cannot authenticate
- **תובנה:** הסטודנט נדרש להפגין הבנה של "מערכת מורשת" (Legacy) ולממש החרגות (Exceptions) בקוד, מיומנות קריטית בצוותי פיתוח מודרניים.

ג. הנדסת מערכת מתקדמת וניהול משאבים

קובץ רלוונטי: simulation_log_20260215_171215.txt

לוג זה מדגים את היכולת של המערכת לדחוף את הסטודנט לפתרונות הנדסיים ברמה גבוהה (Senior Level).

- **הוכחה מהלוג:** בתגובה לחסימת IP (Rate Limit), הסטודנט מימש פתרון מורכב הכולל תכנות אסינכרוני (asyncio), ניהול מכסות (aiolimiter) ושכבת מטמון (aiocache).
- **תובנה:** המערכת מאמתת לא רק את התיקון הלוגי אלא גם את איכות המימוש הטכני והתאמתו לסביבות High-scale.

ד. אימות מנגנון ה-Signature (Anti-Cheat)

השוואה בין הלוגים (למשל 135409 מול 015801) מוכיחה את יכולת ה-Scale: עבור אותה בעיה טכנית (DB Lock), המערכת הזריקה "חתימות" שונות (סימנות כמו DCB_ מול FDF_). הארכיטקט פסל פתרונות שהשתמשו בשמות גנריים, מה שמבטיח שכל סטודנט חייב לפתור את התרחיש הספציפי שלו ללא יכולת העתקה.

ה. ניתוח ויזואלי של מסלול הלמידה (End-to-End Simulation Trace)

מתוך לוג 015801

- **תחנה 1: אתחול וזיהוי (The Setup)**

```
1  ID Please enter your Student ID to begin:
2  Generated scenario for 54325345:
3  {'id': 'db_lock',
4   'name': 'User Data Transfer_FDF',
5   'dev_requirement': "Add a 'last_login_FDF' column to the 'Members_FDF' table.",
6   'prod_issue': " * OUTAGE ALERT: Members cannot log in!\nX The database is unresponsive because the 'Members_FDF' table was locke
d.",
7   'required_fix_concept': 'online_migration',
8   'validation_criteria': 'Migration must use batching or non-blocking syntax to avoid downtime.',
9   'requirements': ['add_column_last_login_FDF', 'batch_updates_or_nonblocking_FDF', 'preserve_user_access_FDF'],
10  'risk_level': 'high'}
```

שלב ה-Personalization: יצירת חתימה הנדסית ייחודית למניעת העתקות

- **תחנה 2: המשימה (The Bait)**

```
1  Agent: Welcome to the Production Trap Simulator!
2  Agent: Team Lead Task:
3  Add a 'last_login_FDF' column to the 'Members_FDF' table.
```

הצגת דרישה פונקציונלית פשוטה לכאורה (הוספת עמודה ב-DB)

- **תחנה 3: ה-Deployment והקריסה (The Trap)**

```
1 Agent: DEPLOYING TO PRODUCTION
2 Agent: 🚨 PRODUCTION ALERT 🚨
```

```
1 ✖ OUTAGE ALERT: Members cannot log in!
2 ✖ The database is unresponsive because the 'Members_FDF' table was locked.
3
```

רגע ה'מלכודת': המעבר המהיר מאישור פיתוח לקריסת מערכת בייצור עקב נעילת טבלה

● תחנה 4: הדרכת הארכיטקט (The Mentorship)

```
1 Agent: The code must include 'batch_updates_or_nonblocking_FDF'
2 to avoid locking the 'Members_FDF' table.
```

מתן הכוונה מושגית ומיקוד ארכיטקטוני, תוך השארת האחריות על המימוש הלוגי והסינטקטי בידי הסטודנט.

● תחנה 5: הסוף המוצלח (The Resolution)

```
1 Agent: SOLVED
2
3 =====
4 🏆 MISSION ACCOMPLISHED! SYSTEM STABLE.
5 =====
6
```

סיום הסימולציה: הסטודנט הטמיע פתרון High-scale והמערכת חזרה ליציבות

11. אתגרים הנדסיים ופתרונות חסינות (Robustness & Integrity)

מעבר להיבטים המבניים והפדגוגיים, קיימים מספר אתגרים הנדסיים מורכבים במימוש המערכת, הדורשים פתרונות חשיבתיים יצירתיים כדי לשמור על יציבות הסימולציה. נושאים אלו מהווים את ה"מנוע" הטכני שמאחורי הקלעים:

1. אתגר זיהום ההקשר (Context Overflow)

בסימולציה איטרטיבית שבה הסטודנט שולח קוד שוב ושוב, היסטוריית ההודעות גדלה במהירות. הדבר עלול להוביל להריגה ממכסת ה-Tokens או לבלבול של המודל.

- הפתרון החשיבתי: מימוש מנגנון Unique Message Filtering באמצעות הפונקציה `add_unique_messages`. המערכת בודקת מזהים ייחודיים (UUID) של הודעות ומוודאת שרק מידע חדש ורלוונטי נשמר ב-State, מה שמונע "ניפוח" מיותר של ההקשר ושומר על דיוק הסוכנים.

2. אתגר ה"לופ האינסופי" (Infinite Loop Prevention)

במערכת Multi-Agent, קיים סיכון שהסטודנט והארכיטקט ייכנסו ללופ אינסופי של תיקונים קטנים מבלי להתקדם לפתרון.

- הפתרון החשיבתי: הגדרת Validation Criteria ברורה לכל תרחיש ב-`scenarios.py`. קריטריונים אלו משמשים כ"תנאי עצירה" דטרמיניסטיים עבור הארכיטקט. ברגע שהקוד עומד ברשימת הדרישות הסגורה, הסוכן מחויב להחזיר את מילת המפתח SOLVED, מה שקוטע את הלופ ומעביר את הגרף למצב סיום.

3. אתגר ה"נפילה השקטה" (Graceful Failure & Robustness)

סוכני LLM עלולים לעיתים להחזיר תשובות בפורמט לא תקין או להיכשל בשל בעיות קישוריות, מה שעלול לעצור את כל חווית הלמידה של הסטודנט.

- הפתרון החשיבתי: הטמעת Error Handling חסין בתוך צמתי הארכיטקט (Architect Node). בקוד קיים בלוק `try-except` העוטף את ניתוח ה-LLM; אם מתרחשת שגיאה טכנית בלתי צפויה, המערכת מוגדרת לבצע "Fail-Open" ולסמן את המשימה כפתורה (SOLVED), כדי להבטיח שהסטודנט לא ייתקע בשל באג במערכת.

4. אתגר העמימות מול הכוונה (The Scaffolding Challenge)

קושי ליצור מערכת שמצד אחד לא נותנת את התשובה (Spoon-feeding) ומצד שני לא משאירה את הסטודנט מתוסכל מדי.

- הפתרון החשיבתי: הנדסת פרומפט המגבילה את הארכיטקט לספק "רק רמז טכני קצר אחד בכל פעם". פתרון זה מייצר "פיגומים פדגוגיים" (Scaffolding) – הסטודנט מקבל מספיק מידע כדי להתקדם שלב אחד, אך עדיין נדרש לבצע את המחקר והמימוש בעצמו.

5. אתגר אמינות הדיווח (Logger Integrity)

כדי שהמרצה יוכל לסמוך על תוצאות הסימולציה, יש לוודא שתיעוד האירועים אמין ובלתי ניתן לזיוף מצד המשתמש.

- הפתרון החשיבתי: שימוש במחלקת ה-Logger המייצרת קובץ עם חותמת זמן ייחודית (TIMESTAMP) בכל הרצה. המערכת מתעדת את כל "מחשבות" הסוכנים וקוד הסטודנט בזמן אמת, מה שיוצר "קופסה שחורה" של תהליך הלמידה.

12. סיכום ומסקנות

פרויקט **Production Trap Simulator** מהווה מימוש מוצלח של סביבת למידה מבוססת AI העומדת בסטנדרטים תעשייתיים. המערכת מצליחה לקחת בעיה תכנותית "גנרית" ולהפוך אותה לאתגר ניהולי וטכני מורכב. השילוב בין חסינות להעתקות, ניהול מצב מתקדם ודיוק פדגוגי מכין את הסטודנטים בצורה מיטבית לעבודה בצוותי פיתוח מודרניים.